

FE-APP-05 - Reporting Portal - Developer Implementation Guide

Version: v1.0 **Bundle:** 99_frontend **Companion DD:** FE-APP-05-Reporting_Portal-v1.0

1. Goal

Implement the reporting portal UX as a consumer of REP-01 APIs.

Start inside the Backoffice shell unless deployment decides to split reporting into a separate app.

2. Runtime Flow

```
sequenceDiagram
    participant UI as Reporting UI
    participant AUTH as Auth/RBAC
    participant REP as REP-01
    participant MOD as Owning Module

    UI->>AUTH: route guard checks reporting permission
    UI->>REP: GET dashboard catalog
    UI->>REP: POST dashboard query with filters
    REP-->>UI: KPIs/charts/table data
    UI->>REP: POST export request
    UI->>MOD: open drilldown read route when user clicks source record
```

3. Step 1 - Routes

Add routes:

- /reporting
- /reporting/:dashboardCode
- /reporting/exports
- /reporting/reconciliation

Use the shared FE-APP-00 layout and Backoffice navigation if embedded in FE-APP-01.

4. Step 2 - Load Dashboard Catalog

Call:

```
GET /api/reporting/dashboard-catalog
```

Render only dashboards returned by REP-01. Do not hardcode access in the frontend.

Expected response shape:

```
{
  "items": [
    {
      "dashboardCode": "FIN_REVENUE",
      "label": "Finance Revenue",
      "group": "Finance",
      "allowedExportFormats": ["CSV", "XLSX"],
      "defaultFilters": {
        "period": "CURRENT_MONTH"
      }
    }
  ]
}
```

5. Step 3 - Query Dashboard

Call:

```
POST /api/reporting/dashboards/FIN_REVENUE/query
Content-Type: application/json
```

```
{
  "filters": {
    "periodFrom": "2026-08-01",
    "periodTo": "2026-08-31",
    "areaCode": "NRB-WEST"
  },
  "viewMode": "SUMMARY"
}
```

The UI renders:

- KPI tiles;
- chart panels;
- table panels;
- freshness marker;
- filter summary.

Do not calculate authoritative values in the browser. Format only.

6. Step 4 - Drilldown

For KPI click:

```
POST /api/reporting/dashboards/FIN_REVENUE/drilldown
Content-Type: application/json
```

```
{
  "metricCode": "OVERDUE_AMOUNT",
  "filters": {
    "periodFrom": "2026-08-01",
    "periodTo": "2026-08-31"
  },
  "page": 1,
  "pageSize": 50
}
```

Rows should include owner:

```
{
  "sourceModule": "BIL-01",
  "sourceRefType": "INVOICE",
  "sourceRefId": "inv-001",
  "display": {
    "customerName": "Jane A.",
    "amountDue": 1200
  }
}
```

Open the owning Backoffice route for details.

7. Step 5 - Export

Call:

```
POST /api/reporting/exports
Content-Type: application/json
```

```
{
  "dashboardCode": "FIN_REVENUE",
  "format": "XLSX",
  "filters": {
    "periodFrom": "2026-08-01",
    "periodTo": "2026-08-31"
  }
}
```

Show status from:

```
GET /api/reporting/exports
```

Download through REP-01 only.

8. Step 6 - UI States

Implement:

- loading state per panel;
- no data state;
- no permission state;
- stale data warning;
- partial panel failure;
- export queued/running/failed/completed;
- row limit exceeded warning.

9. Step 7 - Tests

1. Catalog hides unauthorized dashboards.
2. Dashboard query sends filters correctly.
3. Freshness warning renders.
4. Drilldown uses owning module route.
5. Export request and download flow works.
6. No browser-side KPI recomputation.
7. Row-level permission is enforced by REP response and UI does not override it.